Top-k Per-Flow Cardinality Measurement

[redacted]

1 Introduction

Cardinality constitutes an important statistic of network traffic measurement. Per-flow cardinality measures the number of distinct elements in every flow during the measurement period. A variety of labels can act as flow or element identifiers, such as source address, destination addresses, protocol types, other packet header fields, and even payload keywords. Depending on the identifiers chosen, there are many practical applications of per-flow cardinality. For example, using source IP address to identify flows and destination IP address to tag elements, we can effectively quantify the number of distinct destinations each source talks to, and thus find possible scan attacks.

There has been abundant research on cardinality estimation for a single flow, but the challenge in real-life applications resides in the multi-flow tracking. Particularly, we want to identify the cardinality of top k flows with the highest cardinalities within multiplicative ϵ error. With $ZipFian$ distribution assumption, we developed and analyzed a new algorithm using probabilistic update to achieve $\tilde{O}\left(k\left(\frac{1}{\epsilon^2}\right)^{(1+\frac{1}{\alpha})}\log\log n\right)$ space complexity. Also, we found that the space lower bound for the top-k cardinality estimation problem algorithms when the data is highly skewed is $\Omega(\frac{1}{k\epsilon^2})$.

2 Problem Statement

Assume there are p flows streaming in, each with true cardinality n_i, $1 \leq i \leq p$. Ideally, we want to find the k flows with the highest per-flow cardinality.

- **Space Lower bounds.** We define the (ϵ, δ) top-k flow cardinality problem as follows: given a stream of p flows, for each top k flows with cardinality n_i, the algorithm produces a cardinally estimation within $n_i \pm \epsilon \sum_i n_i$ with probability $1 - \delta$. We would like to obtain a space lower bound for this problem.
- **Top-K Elephant Flows.** Let $f_1, f_2, ...f_k$ be the top k flows with cardinality $n_1, n_2, ..n_k$. We would like to design an algorithm that takes a stream of elements as input, and returns flows $f_1', f_2', ..., f_k'$, so that $(1-\epsilon)n_i \leq \tilde{n_i'} \leq (1+\epsilon)n_i$ simultaneously with constant probability.

3 Related Works

The most naive way of cardinality measurement is to keep a record of all seen elements. However, this increases space linearly with the total cardinality of streaming elements, p. To be more space efficient, many algorithms have been proposed to estimate the single-flow cardinality, including PCSA [4], MinCount [5], LogLog [2] and HyperLogLog [3]. The basic idea of these algorithms is to use multiple registers, bitmaps or other elementary data structures to estimate the cardinality.

HyperLogLog. HyperLogLog [3] is the state-of-art algorithm of cardinality estimation for a single flow. Hyperloglog first constructs m subflows and hashes each item in the flow to one of the subflows uniformly. For each item x in subflow f_j, let $pos(h(x))$ denote the position (from right to left) of the rightmost bit 1 in the hash value $h(x)$. As items stream through, HyperLogLog records the maximum $pos(h(x))$ for each subflow f_j in a register $M[j]$, and estimates the size of the subflow f_j to be $2^{M[j]}$. Finally, HyperLogLog takes the harmonic average over all estimated subflow sizes and outputs the final estimate, which is $\Theta(m)$ times the harmonic average. HyperLogLog uses $O(\frac{1}{\epsilon^2}\log\log n)$ space to achieve ϵ approximation with high probability.

Lower Bounds. In [8], Woodruff proved that $\Omega(\frac{1}{\epsilon^2})$ space is needed to achieve (ϵ, δ) approximation on the single-flow cardinality estimation. To solve for this lower bound, they used reduction from the (ϵ, δ)-HAMMING problem. It is defined as a one way communicate problem where A holds vector x and B holds vector y. Only A is allowed to send information to B. Based on communication from A and input y, B attempts to output an (ϵ, δ) estimation of $|x - y|_1$. It is shown that the (ϵ, δ)-HAMMING problem has communication lower bound $\Omega(\frac{1}{\epsilon^2})$, which yields the lower bound of single flow (ϵ, δ) cardinality estimation problem by reduction.

Virtual Register Sharing. When multiple network flows need to be tracked simultaneously, due to the limited memory, it is impossible to keep a cardinality estimator for every possible flow. Xiao [9] presents a virtual register sharing technique, which enables estimators for multiple flows to share registers. The author deploys this sharing technique on a set of HyperLogLog estimators to track per-flow cardinality and derives a complex variance on per-flow cardinality, which we will analyze in 5.1.

4 Lower Bounds

Let n_i denote the cardinality of the flow f_i and U_i be the universe of possibly incoming elements for the flow f_i. We set the maximum universe size $\max_i |U_i| = n$. In this section, we prove a lower bound of $O(\frac{1}{k\epsilon^2})$ for estimating the cardinalities of the top k flows within additive error $\epsilon \sum_i n_i$ with probability $1 - \delta$ where $\delta < \frac{1}{2}$, under the assumption that $\sum_i n_i = O(kn)$. Since we assume that the top k flows have significantly larger cardinality than smaller flows, and $n \geq |U_i| > n_i, \forall i$, our assumption is justified. (Note that $\sum_i n_i = O(kn)$ is a weaker assumption than the ZipFian distribution we use in section 5 when $\alpha \geq 2$.)

We prove the lower bound for the top-k flow cardinality estimation by reduction from a variant of the GAP-HAMMING problem. We first describe a classic problem called INDEX in 4.1, which has linear one-way communication complexity. Next we describe the GAP-HAMMING problem in 4.2.

The Index problem can be reduced to the GAP-HAMMING problem, which can in turn be reduced to the single-flow cardinality problem. This was used to derive the lower bound of the single-flow cardinality estimation in 4.3. Following this approach, we create a variant of GAP-HAMMING called k-GAP-HAMMING and prove its complexity with reduction from INDEX in 4.4. Finally, we reduce k-GAP-HAMMING to top-k cardinality estimation to prove its lower bound in 4.5.

Note: The complexity proof of the INDEX(4.1), GAP-HAMMING(4.2), and single-flow cardinality(4.3) problems are based upon [7].

4.1 The INDEX Problem

In the INDEX(n) problem, Alice holds a vector $x \in \{0,1\}^n$, and Bob holds an index $i \in [n]$. The one-way communication is from Alice to Bob. The goal is for Bob to output x_i.

Randomized INDEX(n, δ) Alice holds a vector $x \in \{0,1\}^n$, and Bob holds an index $i \in [n]$. Using a randomized public coin communication protocol R, Bob outputs x_i correctly with probability $1-\delta$ where $\delta < \frac{1}{2}$.

Claim 1 [7, p.24-27] Randomized INDEX problem requires $\Omega(n)$ communication from Alice to Bob.

4.2 The GAP-HAMMING Problem

In the HAMMING(n) problem, Alice holds a vector $x \in \{0,1\}^n$, and Bob holds a vector $y \in \{0,1\}^n$. The goal is for Bob to output the hamming distance between x and y, which is the number of indexes where $x_i \neq y_i$ and denoted by $\|x-y\|_0$.

In the GAP-HAMMING(n, ϵ, δ, t) problem, Bob is required to output 1 when $\|x-y\|_0 < t - \epsilon n$ with probability $1-\delta$, and 0 when $\|x-y\|_0 < t + \epsilon n$ with probability $1-\delta$.

In this subsection, we prove by reduction from INDEX that GAP-HAMMING($n, \epsilon, \delta', \frac{n}{2}$) requires $\Omega(\frac{1}{\epsilon^2})$ one-way communication.

Claim: For any $\epsilon \geq \frac{1}{\sqrt{n}}$, Randomized INDEX($\frac{c^2}{\epsilon^2}, \delta + \delta'$) reduces to GAP-HAMMING($n, \epsilon, \delta', \frac{n}{2}$) for some constants $\delta, \delta' < \frac{1}{4}$ and some constant c.

Proof: Alice has $x \in \{0,1\}^{\frac{c^2}{\epsilon^2}}$ and Bob has an index $j \in [n]$. Let $r_1, r_2, ..r_n$ be n unbiased segments of random bits with length $\frac{c^2}{\epsilon^2}$ published by some third party. Given a protocol of GAP-HAMMING($n, \epsilon, \delta, \frac{n}{2}$), one could create a protocol for Random INDEX($\frac{c^2}{\epsilon^2}$) as follows,

- $\forall i$, Alice computes $\|x - r_i\|_0$. If $\|x - r_i\|_0 < \frac{|x|}{2}$, set $a_i = 1$, otherwise, set $a_i = 0$. Let $a = a_1 a_2 ... a_n$ be Alice's input to the GAP-HAMMING problem.
- $\forall i$, let r_{ij} be the jth bit of r_i. Let $b = r_{1j} r_{2j} ... r_{nj}$ be Bob's input to the GAP-HAMMING problem.
- If GAP-HAMMING returns 1, then INDEX returns 1, otherwise INDEX returns 0.

Let $m = |x| = \frac{c^2}{\epsilon^2}$. WLOG, assume m is odd. Now we show that the ith input from a, a_i, and the ith input from b, b_i, are correlated. Denote the number of zeros in r_i excluding the bit r_{ij} ($r_i \setminus r_{ij}$) to be $N(r_i, j)$. If there are $N(r_i, j) < \frac{m-1}{2}$, then $a_i = 0$, if $N(r_i, j) \geq \frac{m+1}{2}$ zeros in $r_i \setminus r_{ij}$, then $a_i = 1$. In both above cases $a_i = b_i$ with probability $\frac{1}{2}$. However, when $N(r_i, j) = \frac{m-1}{2}$ in $r_i \setminus r_{ij}$, then $a_i = \mathbb{1}[x_j \neq r_{ij}]$. Thus when $x_j = 1$, $a_i = \neg b_i$, when $x_j = 0$, $a_i = b_i$.

$$P\left[N(r_i, j) = \frac{m-1}{2}\right] = \binom{m-1}{(m-1)/2} \cdot \frac{1}{2^{m-1}} = c \cdot (1 + \frac{1}{\sqrt{\delta}}) \cdot \frac{1}{\sqrt{m}} \quad \text{(By Stirling's formla)}$$

(Note: use this constant c as c in $|x| = \frac{1}{c^2\epsilon^2}$)

When $x_j = 1, P[a_i = b_i] = P\left[N(r_i, j) \neq \frac{n}{2}\right] \cdot \frac{1}{2} + P\left[N(r_i, j) = \frac{n}{2}\right] \cdot 0 = \frac{1}{2} - \epsilon \cdot (1 + \frac{1}{2\sqrt{\delta}})$

Similarly when $x_j = 0, P[a_i = b_i] = \frac{1}{2} + \epsilon \cdot (1 + \frac{1}{2\sqrt{\delta}})$

Thus

$$\text{when } x_j = 1, E[\|a - b\|_0] = n \cdot (1 - \frac{c}{\sqrt{m}}) = \frac{1}{2}n - \epsilon \cdot (1 + \frac{1}{2\sqrt{\delta}})n$$

$$VAR[\|a - b\|_0] = n\left(\frac{1}{2} - \epsilon \cdot (1 + \frac{1}{2\sqrt{\delta}})\right)\left(\frac{1}{2} + \epsilon \cdot (1 + \frac{1}{2\sqrt{\delta}})\right) \leq \frac{n}{4}$$

$$\sigma[\|a - b\|_0] \leq \frac{1}{2}\sqrt{n}$$

$$\text{when } x_j = 0, E[\|a - b\|]_0 = n \cdot (1 + \frac{c}{\sqrt{m}}) = \frac{1}{2}n + \epsilon \cdot (1 + \frac{1}{2\sqrt{\delta}})n$$

$$\sigma[\|a - b\|_0] \leq \frac{1}{2}\sqrt{n}$$

Therefore by Chebyshev's bound, when $x_j = 1$,

$$P\left[\|a - b\|_0 \geq \frac{1}{2}n - \epsilon n\right] \leq P\left[|\,\|a - b\|_0 - \mu| \geq \frac{1}{\sqrt{\delta}}\sigma\right] \leq \delta$$

Similarly when $x_j = 0$,

$$P\left[\|a - b\|_0 \leq \frac{1}{2}n + \epsilon n\right] \leq \delta$$

Thus, with $1 - \delta$ probability, GAP-HAMMING's input hamming distance $> \epsilon n$ away from $t = \frac{n}{2}$, and in this case GAP-HAMMING returns the correct bit with probability $1 - \delta'$. Thus the protocol results in the correct output with probability at least $(1 - \delta)(1 - \delta') \geq 1 - (\delta + \delta')$.

Since Random INDEX($\frac{c^2}{\epsilon^2}$) requires $\Omega(\frac{1}{\epsilon^2})$ space, so does GAP-HAMMING.

4.3 The Single Flow Cardinality Estimation

In order to bound the space complexity of the single flow cardinality estimation problem, let's first consider the equivalent one-way communication problem:

Let $F_0(n, \epsilon, \delta)$ denote the problem where Alice has an input stream a consisting of multiple elements and Bob has the rest of the input stream b. Let n be the universe size of this flow. Let the cardinality of input flow be F_0. Then Bob needs to output cardinalilty estimation within $(1 \pm \epsilon)F_0$ with probability $1 - \delta$.

Here we show a simple reduction from GAP-HAMMING$(n, \epsilon, \delta, \frac{n}{2})$ to $F_0(n, \frac{\epsilon}{2}, \delta)$. Let $U = \{1, 2, ...n\}$. In the GAP-HAMMING problem, Alice and Bob are given inputs $x, y \in \{0, 1\}^n$. Alice and Bob can convert x, y into $x', y' \subseteq U$ such that x', y' contains all indexes i where $x_i = 1$, $y_i = 1$, respectively. x', y' can then be fed into $F_0(n, \epsilon, \delta)$, which returns F_0 to Bob. In addition, Alice can communicate $|x'|$ in $O(\log n)$ bits to Bob. Finally, Bob returns $\|x - y\|_0^* = 2|F_0| - |x'| - |y'|$ for the GAP-HAMMING problem. This completes the reduction.

The number of unique elements in x', y' is equal to $|x' \cup y'|$.Thus $F_0(n, \epsilon, \delta)$ returns $(1 + \frac{\epsilon}{2})|x' \cup y'|$ with probability $1-\delta$. Observe that $\|x - y\|_0 = 2|x' \cup y'| - |x'| - |y'|$. Thus $\|x - y\|_0^*$ has an additive error within ϵn with probability δ.

Since GAP-HAMMING problem has $\Omega(\frac{1}{\epsilon^2})$ communication complexity, so does $F_0(n, \epsilon, \delta)$.

4.4 The k-GAP-HAMMING Problem

We define **k-GAP-HAMMING(n, ϵ, δ)**: Alice has input $x_1, x_2, ...x_k \in \{0, 1\}^n$, and Bob has input $y_1, y_2, ...y_p \in \{0, 1\}^n$. Bob is required to output an estimate for $\|x_i - y_i\|_0$ with additive error ϵn, for all i, with for-each probability $1 - \delta$.

In this subsection we demonstrate how to reduce INDEX$(\frac{kc^2}{\epsilon^2}, \delta + \delta')$ to k-GAP-HAMMING(n, ϵ, δ).

First let us define another problem **K-INDEX(ϵ, δ)**: Alice has inputs $x_1, x_2, ...x_k$ and Bob has indexes $i_1, i_2, ...i_k \in [n]$. Bob must output an estimate of $x_{ii_1}, x_{2i_2}...x_{ki_k}$, with the probability $1 - \delta$ for each value.

We can easily reduce INDEX(kn, δ) to K-INDEX(n, δ). In the INDEX problem, Alice separates her input x into k segments of strings $\in \{0, 1\}^n$ evenly and generate $x_1, x_2, ..., x_k$. Bob first calculates the segment where his input i belongs by $seg(i) = \lceil \frac{i}{n} \rceil$. Then, Bob can generate random integers in the range of $[(j-1)n, jn), \forall j \neq seg(i)$ as his additional input to K-INDEX. Finally, Bob returns the $seg(i)^{th}$ number out of K-INDEX's output for the INDEX problem, which is correct with probability $1 - \delta$. This completes the reduction.

Since we already knows INDEX$(\frac{c^2}{\epsilon^2}, \delta + \delta')$ reduces to GAP-HAMMING(n, ϵ, δ). Thus K-INDEX$(\frac{c^2}{\epsilon^2}, \delta + \delta')$ reduces to K-GAP-HAMMING(n, ϵ, δ). In conclusion, K-GAP-HAMMING(n, ϵ, δ) has one-way communication complexity $\Omega(\frac{k}{\epsilon^2})$.

4.5 Top k Cardinality Estimation

An obvious lower bound for top-k cardinality estimation would be $\Omega(\frac{1}{k^2\epsilon^2})$ by directly applying the lower bound for single flow carnality estimation. However, we can achieve a tighter bound because of our assumption of the skewed data: $\sum_i n_i = O(kn)$.

The reduction from K-GAP-HAMMING to top k cardinality estimation is similar to the reduction in 4.3. Specifically, we reduce K-GAP-HAMMING$(n, O(k)\epsilon, \delta)$ to the communication version of our top k cardinality estimation problem.

We define the problem $kF_0(n, \epsilon, \delta)$ where Alice holds first half of flows and Bob holds the rest. Among the multiple flows hold by Alice and Bob, the maximum universe size of any flow does not exceed n. Bob has to output an estimate of the top k cardinalities within ϵn additive error with for each probability $1 - \delta$.

Now we show the reduction from K-GAP-HAMMING to kF_0. Alice converts her k input x_i into a stream of elements $\{(i, j)|x_{ij} = 1\}$ where x_{ij} denotes the jth bit of the ith input x_i ($i \in [1, k], j \in [1, n]$). Generally, an element (i, j) denotes a element tagged by j in the flow i. Alice sends all the elements to kF_0. Similarly, Bob converts his k input y_i and inputs his elements to kF_0. Based on our assumption that $\sum_i n_i = O(kn)$, we obtain that $\epsilon \sum_i n_i = O(k)\epsilon n$. Thus, the top k cardinality estimation problem is equivalent to $kF_0(n, O(k\epsilon), \delta)$.

Since there are only k flows, $kF_0(n, O(k)\epsilon, \delta)$ returns individual estimates of $|x'_i \cup y'_i|$, which can be used to estimate $\|x_i - y_i\|$ with the additive error $O(k)\epsilon n$ and $1-\delta$ probability for each pair of inputs x_i, y_i. This completes the reduction from K-GAP-HAMMING$(n, O(k)\epsilon, \delta)$ to $kF_0(n, O(k\epsilon), \delta)$.

As proved in 4.4, K-GAP-HAMMING$(n, O(k)\epsilon, \delta)$ has one-way communication complexity $\Omega(\frac{k}{k^2\epsilon^2}) = \Omega(\frac{1}{k\epsilon^2})$, thus the space complexity of top k cardinality estimation is $\Omega(\frac{1}{k\epsilon^2})$.

5 Algorithms

In this section we describe our algorithm for the top-k cardinality estimation problem, and compare it with Xiao's algorithm of virtual register sharing.

5.1 Xiao's Algorithm : Virtual Register Sharing

5.1.1 Virtual Register Sharing Algorithm

Let the sum over all flow cardinalities be n and the cardinality of a specific flow f_i be n_i. In [9], Xiao creates a table M of m Hyperloglog counters. Then each flow f_i is assigned s virtual registers uniformly at random. Multiple flows $f_1, f_2, ...f_l$ can share the same virtual register $M[j]$ and as such, each time an element from either of flows $f_1, f_2, ...f_l$ comes in, $pos(h(x))$ is computed, and the virtual register $M[j]$'s value is updated to $\max(pos(h(x)), M[j])$. In effect, $2^{M[j]}$ is an estimate for the sum of cardinalities of $f_1, f_2, ..f_l$.

For a specific flow f_j, let its assigned virtual registers be $M[r_i(f_j)]$ for $i = 1, 2, ...s$. Since the function r_i is uniformly at random, the expected sum over cardinalities of flows that are assigned register $r_i(f_j)$ is $n_j + \frac{n-n_j}{m}$. Thus the bias in each register is $\frac{n-n_j}{m}$ and can be corrected. After bias correction, the harmonic average over $2^{M[r_i[f_j]]}$ is taken to obtain a final estimation $\tilde{n_{f_j}}$ of f_j's cardinality.

5.1.2 Space Complexity

[9] calculates that the variance of the above scheme is

$$VAR[\tilde{n_f}] \approx (\frac{m}{m-s})^2 \left(\frac{\beta^2}{s} \left(n_f + (n-n_f)\frac{s}{m} \right)^2 + (n-n_f)\frac{s}{m}(1-\frac{s}{m}) + (\frac{s}{m})^2 \frac{\beta^2}{m} n^2 \right), \quad (1)$$

where β is a constant ≈ 1.04. We know that when $VAR[\tilde{n_f}] \leq O(\epsilon^2)n^2$, then $\sigma(\tilde{n_f}) \leq O(\epsilon)n$. Since the expectation of $\tilde{n_f}$ is n_f, $P[\tilde{n_f} = n_f \pm O(\epsilon)n]$ is a constant. The above expression is very complicated with codependency between s and m. However, we can evaluate this expression by substituting m and s with specific values.

When $s = O(\frac{1}{\epsilon^2})$, $m = O(\frac{1}{\epsilon^2})$ and $m > 2s$, then equation (1) simplifies to

$$VAR[\tilde{n_f}] \leq O(\epsilon^2)n^2 + O(1) \cdot n$$

When $\epsilon \geq \frac{1}{\sqrt{n}}$, $VAR[\tilde{n_f}] \leq O(\epsilon^2)n^2$. This implies that when $\epsilon \geq \frac{1}{\sqrt{n}}$, $O(\frac{1}{\epsilon^2})$ Hyperloglog registers are needed to give an additive ϵn approximation for each flow. However, when $\epsilon \geq \frac{1}{\sqrt{n}}$, this is no longer true. In order to have a good approximation, s and m must not be on the same order of magnitude. Specifically, we substitute in $s = O(\frac{1}{\epsilon^2})$ and $m = O(\frac{1}{\epsilon^4})$, then equation (1) simpilies to

$$VAR[\tilde{n_f}] \leq O(\epsilon^2)n^2 + O(\epsilon^2)n.$$

Therefore, when ϵ is small, as many as $m = O(\frac{1}{\epsilon^4})$ Hyperloglog registers are needed to achieve ϵn additive error with constant probability.

5.2 Our Algorithm: Probabilistic Update

We recall that there are two types of solutions in the top-k frequency problem: counter-based algorithms which maintain counts of only a small portion of the flows, and sketch-based algorithms which records distribution of all the flows [1] [6]. The counter-based algorithms typically use probabilistic updates to find the top-k frequent items.

Inspired by this work, we propose to use the hashing result of an incoming element to probabilistically update the top-k cardinality list. As in [6], we assume the flow cardinality distribution is strongly skewed and has *ZipFian* distribution with $\alpha \geq 1$, which means that given p flows, the kth largest flow cardinality

$$ZipFian(k, \alpha, p) = C \cdot \frac{\frac{1}{k^\alpha}}{\sum_{n=1}^{p}(\frac{1}{n^\alpha})},$$

where C is a constant. Denote the kth largeset flow cardinality as $N(k)$, then since $\alpha \geq 1$, for any β, $N(\beta k) \leq \frac{1}{\beta^\alpha} N(k)$. We use this fact in our algorithm design.

5.2.1 Probabilistic Update Algorithms

We keep s lists, each containing γk Hyperloglog counters. Each list l_i is assigned a different hash function h_i. When the hashing result of an incoming item x yields cardinality larger than the

minimum cardinality in the top-k list l_i, (i.e. $2^{pos(h_i(x))} > \min_{f_j \in l_i} \tilde{n_j}$, where $\tilde{n_j}$ is the cardinality estimate of flow f_j), then flow x is inserted into l_i and its cardinality is recorded as $l_i[f_j] = 2^{pos(h_i(x))}$ using the Hyperloglog counter. If there are already γk elements in l before processing x, then the flow with minimum cardinality is deleted. At any point in the algorithm, if there is a query for the top k cardinality flows, then the algorithm does the following:

- Finds all flows that exists in all of the lists. Let this set of flows be S. if $|S| < k$, report fail.
- Otherwise, for each $f_j \in S$, we compute $\tilde{n_j}$ the harmonic average of $l_i[f]$ over all i, and find the top k flows in S by ranking $\tilde{n_j}$.

5.2.2 Analysis

Let $\zeta = 2\log\frac{1}{\sqrt{s}k}$, . Let $f_1, f_2, ...f_k$ be the top k flows with cardinality $n_1, n_2, ...n_k$. Let $\tilde{n_{ji}}$ denote the cardinality of f_j that would be recorded by l_i if l_i records everything. We prove that when $\gamma = \left(\frac{s}{\delta}\right)^{\frac{1}{\alpha}} \cdot \left(\frac{1}{\zeta}\right)^{\frac{1}{\alpha}}$ and $s = \theta\left(\frac{1}{\epsilon^2}\right)$, then the algorithm returns $f'_1, f'_2, ..., f'_k$, so that $(1-\epsilon)n_i \leq \tilde{n'_i} \leq (1+\epsilon)n_i$ simultaneously with constant probability (proportional to $(1-\delta)$) and uses $O\left(\left(\frac{\epsilon^2}{\delta}\right)^{\frac{1}{\alpha}} \cdot \left(\frac{1}{2\log\frac{1}{\epsilon k}}\right)^{\frac{1}{\alpha}} \epsilon^2 k\right)$ registers.

Denote a flow f's actual cardinality rank as $rank(f)$ (e.g. $rank(f) = k$ means that f has the kth largest cardinality). Since each l_i records the top γk values, there exist flows $f_j^* \in l_i$ with $rank(f_j^*) \leq \gamma k$. By the *ZipFian* distribution, $n_{ji}^* \leq N(\gamma k) \leq \frac{1}{\gamma^\alpha} N(k) = \left(\frac{\delta}{s}\right) \cdot \zeta \cdot N(k)$.

Claim 1: For all list l_i, $\min_{f_j \in l_i} \tilde{n_{ji}} \leq \left(\frac{1}{\zeta}\right) N(k)$ simutaneously with probability $1-\delta$

Proof: We first prove that for any list l_i, $n_{ji}^* \geq \frac{1}{\zeta} N(k)$ with small probability. For any data x, probability that the estimated cardinality $2^{pos(h(x))}$ is $> N$ is as follows,

$$P[2^{pos(h(x))} \geq N] = P[pos(h(x)) \geq \log N] = 2^{\log N} = \frac{1}{N}$$

Since we can bound n_{ji}^*, we know that

$$P\left[\tilde{n_{ji}^*} \geq \left(\frac{1}{\zeta}\right) N(k)\right] \leq 1 - \left(1 - \frac{\zeta}{N(k)}\right)^{\left(\frac{\delta}{s}\right) \cdot \frac{N(k)}{\zeta}} \leq \frac{\delta}{s}$$

Therefore the minimum cardinality in l_i

$$\min_{f_j \in l_i} \tilde{n_{ji}} \leq n_{ji}^* < \left(\frac{1}{\zeta}\right) N(k)$$

with probability $\geq (1 - \frac{\delta}{s})$. Thus the property above holds for all the s lists with probability $\geq (1-\frac{\delta}{s})^s \geq 1 - \delta$

Claim 2: Assume for all list l_i, $\min_{f_j \in l_i} \tilde{n_{ji}} \leq \left(\frac{1}{\zeta}\right) N(k)$, then all flows with top k cardinalities are included in all lists simutaneously with probability at least $(1 - \frac{1}{k})$.

Proof: Let f_j be a flow with top k cardinality. Then $n_j \geq N(k)$. If f_j is not in list l_i, then $\tilde{n_{ji}} \leq \left(\frac{1}{\zeta}\right) N(k)$. The probability that this happens is

$$P\left[\tilde{n_{ji}} \leq \left(\frac{1}{\zeta}\right) N(k)\right] \leq \left(1 - \frac{\zeta}{N(k)}\right)^{N(k)} \leq e^{\zeta} = e^{2\log\frac{1}{\sqrt{sk}}} = \frac{1}{sk^2}$$

Thus the probability that f_j is included in all the lists is

$$P[\forall i, f_j \in l_i] \geq (1 - \frac{1}{sk^2})^s \geq 1 - \frac{1}{k^2}$$

The probability that all top k cardinality flows f_j are included in all the list is

$$P[\forall i, \forall j \leq k, f_j \in l_i] \geq (1 - \frac{1}{k^2})^k \geq 1 - \frac{1}{k}$$

Claim 3: Let $f_1, f_2, ...f_k$ be the top k flows with cardinality $n_1, n_2, ...n_k$. The algorithm returns $f'_1, f'_2, ..., f'_k$, so that $(1-\epsilon)n_i \leq \tilde{n'_i} \leq (1+\epsilon)n_i$ with constant probability.

Proof: By claim 1 and 2, with probability at least $(1-\delta)(1-\frac{1}{k})$, the top k cardinality flows are all included in all lists and hence included in the set S. Since $\forall f_j \in S$, an estimation of f_j's cardinality is recorded in each list without deletion, we can directly use the result from [3], where the harmonic average over $s = \theta(\frac{1}{\epsilon^2})$ Hyperloglog counter estimations gives for all f_j that $(1-\epsilon)n_j \leq \tilde{n_j} \leq (1+\epsilon)n_j$ with high probability each, and thus with constant probability simutanously. In this case, if $f'_j \neq f_j$, WLOG, assume $n'_j \leq n_j$, then for some $i \leq j$, $\tilde{n_i} \leq \tilde{n'_j}$. Thus

$$(1-\epsilon)n_j \leq (1-\epsilon)n_i \leq \tilde{n_i} \leq \tilde{n'_j} \leq (1+\epsilon)n'_j \leq (1+\epsilon)n_j.$$

5.3 Comments

From our analysis, the number of Hyperloglog counters used by our algorithm is $\tilde{O}(k\left(\frac{1}{\epsilon^2}\right)^{(1+\frac{1}{\alpha})})$, which is larger than that of Xiao's algorithm when $\epsilon > \frac{1}{\sqrt{n}}$. However, our algorithm gives a ϵ multiplicative approximation instead of the additive ϵn approximation given by Xiao. We are not sure if our analysis is tight, or if tweaking the algorithm can result in better space complexity. In general, our algorithm is much easier to analyze than Xiao's algorithm, which involves a complicated variance formula.

6 Reflection and Future Work

- Due to time constraint, we did not have a chance to analyze the space complexity of using compressed sensing to recover top k cardinality flow. In the future, we would like analyze the performance of the following compress sensing scheme:

  We use the same virtual register sharing scheme as Xiao, but estimate all flow cardinalities simultaneously by using compressive sensing instead of estimating the individual flow cardinalities separately with harmonic averaging. Intuitively, in Xiao's approach, each register's value can be represented as a linear combination of individual flow cardinalities. Let m be

the number of registers in the table, then we can represent the relationship between the observed register values and the individual flow cardinalities in the form of $Ax = M$, where x_i is the true cardinality of flow f_i, $M[i]$ is the value in register i, and $A[i][j] = 1$ iff f_i uses j^{th} register. Assume the input stream follows a skewed distribution where at most k flows have high cardinality and all the other flows have cardinality significantly smaller. Then x_i is close to k-sparse and we can apply the compressive sensing technique described in lecture 21 to solve the underdetermined linear system $Ax = M$.

- We would like to improve the analysis and the algorithm for our current probabilistic update algorithm.

- We attempted to analyze Xiao's algorithm when the minimum of s registers are taken instead of correcting bias and then taking the harmonic average of s registers. Our hope was that this would simplify the analysis. However, we encountered difficulty calculating the bias of the cardinality estimation. Specifically, consider flow f, let $X_1, X_2, ...X_s$ as the estimated cardinality value of the registers assigned to f. Then we would like to know the expected value of $min(X_1, X_2, ...X_s)$. Since $X_1, X_2, ...X_s$ have independent identical distribution (iid), the cumulative distribution function(c.d.f.) of $min(X_1, X_2, ...X_s)$ can be calculated if we know the c.d.f F of X, the estimated cardinality value of a register, given that the register is assigned to flow f.

  $$F(min(X_1, X_2, ...X_s)) = 1 - (1 - F(x))^n.$$

  However, we did not know how to obtain the c.d.f of X. We can consider X as the sum of independent but not identical variables. Let R be a register assigned to flow f. For each flow f_i, let $\lambda_i = n_i$ the cardinality of f_i if register R is assigned to f_i, and 0 otherwise. Then the estimated cardinality value X recorded by R is

  $$X = \sum_i \lambda_i.$$

  Sum of independent random but not identical variable's c.d.f is complicated to obtain (involve multiple integrations) and it depends on the distribution of cardinalities. We wonder if there is an easier way to calculate the bias.

References

[1] Ran Ben-Basat, Xiaoqi Chen, Gil Einziger, and Ori Rottenstreich. Efficient measurement on programmable switches using probabilistic recirculation. *2018 IEEE 26th International Conference on Network Protocols (ICNP)*, pages 313–323, 2018.

[2] Marianne Durand and Philippe Flajolet. Loglog counting of large cardinalities. In *European Symposium on Algorithms*, pages 605–617. Springer, 2003.

[3] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm. In *Discrete Mathematics and Theoretical Computer Science*, pages 137–156. Discrete Mathematics and Theoretical Computer Science, 2007.

[4] Philippe Flajolet and G Nigel Martin. Probabilistic counting. In *Foundations of Computer Science, 1983., 24th Annual Symposium on*, pages 76–82. IEEE, 1983.

[5] Frédéric Giroire. Order statistics and estimating cardinalities of massive data sets. *Discrete Applied Mathematics*, 157(2):406–427, 2009.

[6] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. Efficient computation of frequent and top-k elements in data streams. In *International Conference on Database Theory*, pages 398–412. Springer, 2005.

[7] Tim Roughgarden et al. Communication complexity (for algorithm designers). *Foundations and Trends® in Theoretical Computer Science*, 11(3–4):217–404, 2016.

[8] David Woodruff. Optimal space lower bounds for all frequency moments. In *Proceedings of the fifteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 167–175. Society for Industrial and Applied Mathematics, 2004.

[9] Qingjun Xiao, Shigang Chen, You Zhou, Min Chen, Junzhou Luo, Tengli Li, and Yibei Ling. Cardinality estimation for elephant flows: A compact solution based on virtual register sharing. *IEEE/ACM Transactions on Networking*, 25(6):3738–3752, 2017.